Normalization, Keys and Constraints

This document checks the 13 PostgreSQL tables of the AdaLink schema against the normal forms, explains every dependency that departs from them and why it is kept, and shows how each business rule is enforced. The design was derived from a normalised ER diagram, so no normalisation actions were needed; the analysis below confirms this and records the deliberate exceptions.

1. Approach

For each table, the functional dependencies (FDs) were listed from the business rules: X → Y means that a value of X always determines a single value of Y. The candidate keys are the minimal sets of columns that determine every other column. Each table was then tested against:

Normal form	Test applied
1NF	Every column holds a single, atomic value; there are no repeating groups.
2NF	1NF, and no non-key column depends on only part of a candidate key.
3NF	2NF, and no non-key column depends on another non-key column (no transitive dependency).
BCNF	Every determinant (the left side of every FD) is a candidate key.

2. Summary

Table	Candidate keys	Normal form	Note
state	state_code; state_name	BCNF	
customer	customer_id; email; phone_num	BCNF	
address	address_id	BCNF	City does not determine state (section 4.5)
vendor	vendor_id; business_name; email_address	BCNF	
warehouse	warehouse_id; warehouse_code	BCNF	
category	category_id; category_name	BCNF	
product	product_id; sku_code	BCNF	Commission kept in category (section 4.4)
warehouse_stock	(warehouse_id, product_id)	BCNF	
order_item	(order_id, product_id)	BCNF	Look-alike columns (section 4.4)
payment	payment_id; payment_reference	BCNF	
shipment	shipment_id; tracking_num; order_id	BCNF	
orders	order_id	2NF	Deliberate: customer_id kept (section 4.1)
payout	payout_id; (vendor_id, start_date)	1NF	Deliberate: end_date and net_amount kept (section 4.2)

11 tables reach BCNF. orders and payout each store one or two calculated or repeated columns on purpose; removing those columns would bring both to BCNF. In both cases a constraint guarantees the stored value can never disagree with the value it repeats, so the usual update anomalies cannot occur.

3. Functional dependencies, table by table

For brevity, "all" means every other column in the table.

Table	Functional dependencies	Analysis
state	state_code → all; state_name → all	Both determinants are candidate keys. The region is stored once per state, not repeated in every address.
customer	customer_id → all; email → all; phone_num → all	Three candidate keys (rule E1: no two customers share an email or phone). Every determinant is a key.
address	address_id → all	Single-column key, so 2NF is automatic. No non-key column determines another (see 4.5 for city and state).
vendor	vendor_id → all; business_name → all; email_address → all	phone_number is not unique, so it is not a key and determines nothing.
warehouse	warehouse_id → all; warehouse_code → all	The code (e.g. LAG-01) is treated as one atomic identifier; its prefix is a naming convention, not data the database relies on.
category	category_id → all; category_name → all	Every determinant is a key.

product	product_id → all; sku_code → all	product_name is not unique (two vendors may sell the same item). The commission rate is not stored here (see 4.4).
warehouse_stock	(warehouse_id, product_id) → qty_on_hand, reorder_level, updated_stock_level_date	All three columns describe a product in a particular warehouse (rule 8), so they depend on the whole key.
order_item	(order_id, product_id) → quantity_ordered, unit_price, commission_rate, payout_id	Each column depends on the whole key: the same product can be sold at different prices and rates, and paid out in different weeks, in different orders.
payment	payment_id → all; payment_reference → all	amount often equals the order total but does not have to (a refund or a part payment), so there is no dependency on order_id.
shipment	shipment_id → all; tracking_num → all; order_id → all	order_id is a candidate key because each order has at most one shipment (rule 14).
orders	order_id → all; address_id → customer_id	address_id is not a key of orders, so customer_id depends on it transitively: not 3NF (section 4.1).
payout	payout_id → all; (vendor_id, start_date) → all; start_date → end_date; (gross_amount, commission_amount) → net_amount	end_date depends on start_date, which is only part of the key (vendor_id, start_date): not 2NF. net_amount depends on two non-key columns: not 3NF (section 4.2).

4. Issues found and design decisions

No table needed to be split. Each point below is either a deliberate, controlled exception or something that looks like a normalisation problem but is not one.

4.1 orders: customer_id depends on address_id (transitive dependency)

Every address belongs to exactly one customer (rule 1), so address_id → customer_id also holds inside orders. Strictly, customer_id could be removed and found through the address, which would make orders BCNF. It is kept because:

- The ER diagram has two separate relationships, places (rule 9) and delivered_to (rule 10); each maps to its own foreign key.
- The composite foreign key (address_id, customer_id) → address(address_id, customer_id) means the two values can never disagree. Tested: changing the owner of an address that an order uses is rejected, so no update anomaly can arise.
- "All orders of a customer" is the most frequent query (order history, support, reports) and needs no join.

4.2 payout: end_date and net_amount are calculated from other columns

start_date → end_date (always six days later, rule 17) and (gross_amount, commission_amount) → net_amount (rule 19). Removing both columns would make payout BCNF. They are kept because a payout is a financial record: the bank transfer, the vendor's statement and an auditor all need the exact figures and period as paid, written down. CHECK constraints (end_date = start_date + 6; net_amount = gross_amount - commission_amount) make an inconsistent row impossible. Tested: changing end_date or net_amount on its own is rejected.

4.3 Totals stored in one table and calculable from another

orders.total_amount equals the order's lines plus its delivery fee (rule 12), and payout.gross_amount and commission_amount equal the sales it settles (rule 19). These are not dependencies inside one table, but they are redundant across tables. They are kept for speed (they are read on every order screen, invoice and payout statement) and, for payouts, because the amount paid must not change if an order line is later edited. Their consistency is checked by verification queries rather than by a constraint.

4.4 Columns that look like duplicates but are not

- order_item.unit_price vs product.unit_price. product.unit_price is today's price; order_item.unit_price is the price actually paid in that order. Prices change, so the second cannot be derived from the first: it is a separate historical fact.
- order_item.commission_rate vs category.commission_rate. The same reasoning: the category's rate can change, and each sale keeps the rate that applied on the day.
- The commission rate is not stored in product. product_id → category_id and category_id → commission_rate would make it transitive. Keeping the rate only in category avoids that dependency.

4.5 address: city does not determine state

It can look as if city → state_code, which would be a transitive dependency needing a separate City table. It does not hold in Nigeria: several place names exist in more than one state, for example Surulere (Lagos and Oyo) and Ifelodun (Kwara and Osun). Both columns are therefore independent facts about the address.

4.6 First Normal Form and the PostgreSQL/MongoDB split

Every PostgreSQL column holds one atomic value. The ER diagram's only multivalued attribute, {specification} of Product, would break 1NF as a column; in a purely relational design it would become its own table (product_id, spec_name, spec_value). In the AdaLink split design it is stored in MongoDB instead, as an embedded object in each product document,

together with reviews and shipment tracking events. None of these appear in the PostgreSQL tables, so they are outside this analysis.

5. How each business rule is enforced

PG = PostgreSQL constraint or trigger; MDB = MongoDB validator or index; App = the application, because the rule spans both databases or cannot be expressed as a constraint.

Rule	Enforced by	Where
E1	Customer ID primary key; email and phone UNIQUE; email format and +234 phone format CHECK	PG
E2	Address ID primary key; addressLine, city NOT NULL; is_default true/false	PG
E3	State code primary key; state name UNIQUE; region CHECK against the six zones	PG
E4	Vendor ID primary key; business name and email UNIQUE; phone format CHECK	PG
E5	Category ID primary key; name UNIQUE; commission rate CHECK 0 to 50	PG
E6	Product ID primary key; SKU UNIQUE; unit price CHECK > 0. Specifications: object of unique names with text values	PG + MDB
E7	Warehouse ID primary key; warehouse code UNIQUE	PG
E8	Order ID primary key; status CHECK against six values; delivery fee CHECK >= 0	PG
E9	Payment ID primary key; method and status CHECK; amount > 0; reference UNIQUE; successful needs date paid; date paid not before date made	PG
E10	Shipment ID primary key; tracking number UNIQUE; delivery date not before dispatch date	PG
E11	Event fields required; status from seven values; time in HH:MM format	MDB
E12	Payout ID primary key; status CHECK; bank reference UNIQUE; a paid payout needs a date and reference	PG
1	address.customer_id NOT NULL foreign key	PG
2	Partial unique index ux_address_one_default_per_customer	PG
3	state_code NOT NULL foreign key in address, vendor and warehouse	PG
4, 5	product.vendor_id and product.category_id NOT NULL foreign keys	PG
6	Not implemented: the ER diagram has no category hierarchy (limitation)	—
7	order_item.commission_rate copied from the category when the sale is made; CHECK 0 to 50	App + PG
8	warehouse_stock primary key (warehouse_id, product_id); qty_on_hand CHECK >= 0	PG
9	orders.customer_id NOT NULL foreign key	PG
10	Composite foreign key (address_id, customer_id) to address	PG
11	order_item primary key (order_id, product_id); quantity >= 1. At least one product per order: application	PG + App
12	Total calculated at checkout; checked by a verification query	App
13	payment.order_id NOT NULL foreign key	PG
14	shipment.order_id NOT NULL UNIQUE foreign key; shipment.warehouse_id NOT NULL foreign key	PG
15	Events embedded in their shipment's document, so each belongs to exactly one shipment	MDB
16	Unique index (customer_id, product_id); rating 1 to 5. Delivered-order check against PostgreSQL before insert	MDB + App
17	payout.vendor_id NOT NULL foreign key; UNIQUE (vendor_id, start_date); CHECK end_date = start_date + 6	PG
18	Nullable order_item.payout_id foreign key; trigger trg_sale_payout checks the vendor and that the order is delivered	PG
19	CHECK net_amount = gross_amount - commission_amount; gross and commission checked by a verification query	PG + App

PostgreSQL totals: 13 primary keys, 17 foreign keys, 14 unique constraints, 28 check constraints, 1 partial unique index and 1 trigger.